Counters Beat a Transformer on a Weak CPU: First Grove-CM Results

Author: Nikunj Mehndiratta. 30 October 2026.

Abstract

Grove-CM is a proposed English language model that uses no dense weight matrices and is built to train and run on CPUs. Its language core is a context-mixing model: hashed counters and bit histories combined by one small mixer that learns online, byte by byte. This report gives the first measurements behind that design. Off-the-shelf context-mixing and statistical compressors served as language models. They were scored on held-out text in bits per byte, against a byte-level transformer trained on the same CPU.

On a free Google Colab CPU runtime, every compressor beat the transformer on both rungs tested:

Rung	Best compressor	Its score	Its time	Transformer score	Transformer time
Rung 0: tiny Shakespeare	paq8px	**1.720**	11 minutes	2.971	3 minutes
Rung 1: Wikipedia (enwik8, 20 MB training text)	lpaq1	**1.553**	2 minutes	2.44	60 minutes

lpaq1 reached its score in a thirtieth of the transformer's time. By the plan's gate rule, the counters climb past both rungs.

The result is real but narrow. It was measured on a weak CPU (one physical core, 47 GFLOP/s of matrix arithmetic), from one run of each model, with an undertrained transformer. The compressors also kept learning on the test text. It says nothing yet about generating coherent English, which is the next rung.

1. Why this experiment

The earlier research report (*Weight free CPU language models*, "Mix Counters, Not Matrices, for Laptop English") surveyed every model family that avoids dense weights:

- weightless lookup-table networks;
- logic-gate networks;
- Tsetlin machines;
- hyperdimensional computing;
- lookup and memory layers;
- retrieval;
- brain-inspired learning;
- context-mixing compressors.

Only context mixing has a benchmark-grade language result without dense neural weights. The Hutter Prize record, about 0.80 bits per character on 1 GB of Wikipedia, was set under rules of one CPU core, no GPU and under 10 GB of RAM.

The report therefore proposed **Grove-CM**: a byte-level context-mixing language model inside the team's existing pieces. Those pieces are a spelling corrector, an intent router and reply bank, and Grove, which holds conversation state and computes every number exactly. The report also set a ladder of tests, each judged against a transformer given the same CPU and the same wall-clock time:

1. Shakespeare
2. Wikipedia
3. TinyStories coherence

4. BabyLM grammar
5. Dialogue

This report covers W0, a measurement of the machine, and the first two rungs.

2. Setup

2.1 Hardware (W0)

The runs used a free Colab CPU runtime. The same probe was run on the team's laptop for comparison.

	Colab free CPU	Laptop (Core Ultra 7 255U, plugged in)
Processor	Intel Xeon @ 2.20 GHz, 2 logical cores (one physical core)	2 performance, 8 efficiency, 2 low-power cores
Vector instructions	AVX2, FMA; no AVX-512	AVX2, AVX-VNNI; no AVX-512
RAM	12.7 GB	16 GB
PyTorch threads	1	12
fp32 matmul, 1024^2 / 2048^2	47 / 38 GFLOP/s	432 / 446 GFLOP/s
Counter updates on one core (rough)	3.5 million/s	10.4 million/s
Training a 19M-parameter transformer stack	378 tokens/s	2,209 tokens/s

The Colab CPU is about three times slower per core than the laptop and six to eleven times slower at neural-network arithmetic. Results below are comparable only within the Colab table.

2.2 Models

Compressors used as language models. The build and command lines follow each project's own documentation.

- **paq8px**, level -8 (about 2.4 GB of memory): a strong context-mixing compressor with hundreds of models.
- **lpaq1**, memory level 9 (about 1.5 GB): a "lite" context mixer, much faster.
- **PPMd**, through 7-Zip (1 GB of memory, model order 10): prediction by partial matching.
- **lzma**, preset 9 extreme: a dictionary compressor, included as a weak reference.

The transformer baseline is a byte-level GPT in PyTorch, trained on the CPU with these settings:

- fp32, no dropout; dropout's random-number generation took 35–41% of a training step in our earlier profile;
- AdamW, a warm-up followed by a cosine schedule over the wall-clock budget;
- PyTorch's default threading.

Rung 0 used nanoGPT's CPU Shakespeare shape: 4 layers, width 128, context 64 bytes, 867,072 parameters. Rung 1 used 4 layers, width 256, context 128 bytes, about 3.3 million parameters.

2.3 Scoring

A compressor that learns online is a language model: its output size is the sum of minus log-probabilities of every byte. Held-out scores come from two runs:

```
bits on test = 8 × (size(train + test) − size(train)), and test bits per byte = bits on test ÷ test length
```

The test text is thus predicted after the compressor has read all the training text. The two runs go in parallel, one per core, and archive headers mostly cancel in the subtraction.

The transformer is scored on the same test text with half-overlapping windows, so every byte after the first window has at least half a window of context. All scores are bits per byte; for this mostly-ASCII text that is effectively bits per character.

2.4 Data

- **Rung 0: tiny Shakespeare** (1.1 MB). The first 90% is training text and the last 10% is test text, nanoGPT's split.
- **Rung 1: enwik8** (the first 100 MB of English Wikipedia). The standard split is 90 MB train, 5 MB validation and 5 MB test. For time, this "quick" rung used only the first 20 MB of the training split, with the standard 5 MB test.

2.5 Fairness rules and their exceptions

The plan gives each transformer the same wall-clock time on the same machine as the strongest compressor. Two exceptions applied:

- **Rung 0:** the transformer had a fixed 3 minutes, matching nanoGPT's CPU baseline. paq8px used 11 minutes; lpaq1 and PPMd used seconds.
- **Rung 1:** paq8px was dropped because it ran at about 1.7 KB/s on this CPU, roughly 4 hours for the 25 MB. The transformer instead received a fixed 60 minutes, thirty times what lpaq1 needed. Both exceptions favour the transformer.

3. Results

3.1 Rung 0: tiny Shakespeare

Model	Test bits per byte	Wall time	Gate (vs transformer)
paq8px -8	**1.720**	660 s	climbs (−1.25)
PPMd	1.859	1 s	climbs (−1.11)
lpaq1 9	1.910	9 s	climbs (−1.06)
lzma	2.518	2 s	climbs (−0.45)
Transformer, 867k parameters	2.971	3 min training	—

The parity target was 2.71, nanoGPT's CPU result. All four compressors met it; the transformer did not. On this CPU it trained on only about 5,000 bytes per second, so in 3 minutes it saw under a million bytes.

3.2 Rung 1: Wikipedia (enwik8, 20 MB of training text)

Model	Test bits per byte	Wall time	Gate (vs transformer)
lpaq1 9	**1.553**	2 min	climbs (−0.89)
PPMd	1.682	under 1 min	climbs (−0.76)
lzma	1.965	1 min	climbs (−0.48)
Transformer, 3.3M parameters	2.44	60 min training, plus test scoring	—
paq8px -8	not run	about 4 hours estimated	—

The transformer's training loss fell steadily:

Time	Training bits per byte
1 min	4.98
5 min	3.79
10 min	3.55
14 min	3.29
60 min	2.3–2.6

It trained at about 1,710 bytes per second, so in 60 minutes it saw roughly 6 MB, under a third of the training text.

The ladder's absolute targets were 1.34 bits per byte for parity with small LSTMs, 1.23 for credibility (cmix) and 1.11 as a stretch. No model reached them with 20 MB of training text, which is expected: those figures come from the full 90 MB split and much stronger models.

3.3 Time to quality

The clearest number is time. lpaq1 reached 1.553 bits per byte in 2 minutes on one core. The transformer was at about 2.4 after 60 minutes on the same CPU.

At the transformer's measured learning curve, it would need many hours on this machine to reach lpaq1's level, if it got there at all. That is an extrapolation, not a measurement.

4. What the result shows, and what it does not

What it shows. On a CPU that is weak at matrix arithmetic, counting-based language models learn English byte statistics far faster than a transformer. This is the hardware the Grove-CM design targets: a one-core free runtime and a 15-watt laptop alike.

- The compressors update a few hashed counters per bit and never multiply a weight matrix.
- They learn from every byte exactly once.
- They need no separate training phase.

This supports the report's central bet that the language core of a CPU-native model should be counters, not matrices.

What it does not show.

1. **Generation.** Compression measures prediction, not the quality of generated text. Sampled text from strong context mixers is known to fall into repetitive loops unless learning is frozen. Coherent generation is exactly what rung 2 (TinyStories) tests, and the earlier report predicts the counters will struggle there.
2. **A fair fight on a strong CPU.** The transformer was undertrained, seeing a third of the rung 1 text. The Colab CPU penalises matrix arithmetic far more than counter updates, and on the laptop (about nine times more GFLOP/s) the gap should narrow. On GPU-class hardware the transformer would win, as the published literature shows; this test is deliberately CPU-only.
3. **Equal treatment of the test text.** The compressors keep learning while they read the test text, an advantage known as dynamic evaluation. The transformer is frozen during scoring.
4. **Statistical certainty.** Each model was run once, with one seed, on one machine. The gaps are large (0.45 to 1.25 bits per byte), so the ranking is unlikely to flip, but no confidence intervals were measured.
5. **Grove-CM itself.** These are off-the-shelf compressors. The dialogue-aware context models, the frozen base with a per-conversation cache, and the links to Grove are not built yet.

5. Deviations from the plan

- **Drive was not used.** The notebook kept results inside the Colab session, at the author's request. The transformer's Drive checkpoints (resume after a disconnect) were therefore unavailable; the run completed without a disconnect.
- **The first attempt was discarded.** It was interrupted while waiting for Drive access and while building paq8px. The scores from that attempt (lzma 2.518, PPMd 1.859, lpaq1 1.910 on Shakespeare) matched the final run exactly.
- **Rung 1 skipped paq8px** and gave the transformer a fixed 60 minutes instead of a matched budget (Section 2.5).
- **Rung 1 used 20 MB, not 90 MB,** of training text.

6. Next steps

1. **Repeat rungs 0 and 1 on the laptop.** Run on mains power with paq8px and the full 90 MB split. The laptop's faster cores make paq8px practical (estimated at about 3× Colab's speed), and its stronger matrix arithmetic gives the transformer a fairer budget.
2. **Rung 2, generation.** Train on TinyStories and sample from the counter model with learning frozen during generation. Have the samples judged for grammar, consistency and creativity against a same-time transformer. This is the rung where the design is most likely to fail, and its kill rule (consistency of 3 out of 10 or below while the transformer reaches 6) decides whether a small neural component is needed.
3. **Start the Grove-CM engine.** Begin with a byte-level context mixer that exposes per-byte probabilities and can freeze learning. Then add the dialogue-context models: previous user turn, intent and persona.

Appendix: reproducing the run

- Notebook: `gcm/colab_gcm.ipynb`. Generate it with `python build_notebook.py` in `gcm/`. Upload it to Colab, choose a CPU runtime and Run all. `USE_DRIVE = False` keeps results in the session; `RUNG1_TRAIN_MB = 90` selects the full enwik8 split.
- Code: `gcm/gcm/` contains the dataset splits, compressor wrappers and held-out scoring, the byte transformer, and the hardware probe. `gcm/tests/` holds six tests, all passing locally.
- Raw results: `gcm/results/colab_2026-10-30.md`.